

Final Project

Donald Pellerin

Cloudcore Customer Churn

A. Background

CloudCore is a mid-sized software-as-a-service (SaaS) company with roughly 12,000 small- to medium-sized enterprise (SME) customers. Customer Churn is up about 15% over the last two quarters, mostly in the basic and mid-tier subscription plans. Together, they make up approximately 70% of the total customer base. Early signals point to thin feature differentiation, onboarding friction, support gaps, and pressure from AI-enabled competitors. Reducing customer churn is the fastest, lowest-risk way to improve CloudCore's customer lifetime value (LTV), reduce customer acquisition cost (CAC), and therefore improve profitability.

Today, CloudCore's analytics team does most of its data analysis using single Excel or CSV table keyed by customerID. The dataset contains customer-level metrics such as tenure, monthly charges, and total charges – and supports basic descriptive analysis. However, because it is a static excel file and not connected to new live data, it hampers the team's ability to track behavior, or identify churn risks in real time.

B. Data Sources

Dataset. 7,043 rows $\times$ 21 columns at the customer level (keyed by customerID).

Missingness is $\sim 0\%$ across columns.

It includes numerical fields (tenure, Monthly Charges, TotalCharges), binary flags (e.g., paperless billing, phone service), as well as categorical service and contract fields (e.g., internet service, security/backup, tech support, streaming, contract type, payment method). In practice, most analysis has been descriptive and reactive, focused on explaining churn after cancellations. The goal now is to shift this analysis toward the earlier detection of churn risk and proactive retention strategies.

Distributions.

- **Tenure** shows a bimodal pattern—a large group of new customers and another group with long tenure, with fewer in the middle.
- **MonthlyCharges** and **TotalCharges** run roughly \$20–\$120 with a visible spike at the low end, and a few high-charge outliers (see box plot).

Observed drivers (from the bar charts).

- **Contract:** Month-to-month $\approx 42.7\%$ churn, 1-year $\approx 11.3\%$, 2-year $\approx 2.8\%$.
- **Payment method:** **Electronic check** is highest at $\approx 45.3\%$ (vs. mailed check $\approx 19.1\%$, bank transfer $\approx 16.7\%$, credit card $\approx 15.2\%$).

Assessment.

This CSV is solid for a **baseline churn read**—tenure, contract, pricing, and payment fields all relate to churn. The gaps are **product-usage telemetry**, **onboarding timestamps**, and **support metrics**, and the file is a **static export**, which limits early-warning and ongoing monitoring.

C. Data Needs

To improve predictive accuracy and to enable proactive retention strategies, CloudCore should expand its current dataset to also include behavioral, operational, and support-based metrics. These three additional data streams would provide deeper visibility into customer engagement and help anticipate churn risk earlier.

- **Product usage telemetry:** Logins per week, active days, projects created, teammates invited, key feature use, and integrations or API calls. These metrics would create **engagement signals** and help detect **usage drops** early.
- **Onboarding funnel events:** Time to first login, first project, first teammate added, and first integration. Tracking these stages would help identify surface friction and measure time-to-value.
- **Support and customer success metrics:** Tickets, severity, first-response time, customer satisfaction (CSAT), and Net Promoter Score (NPS). These data points would help quantify service pain as a churn driver.
- **Renewal/contract metadata:** Auto-renew dates, notice windows, seat counts. Capture this data would support timely outreach enable save plays prior to renewal.

Together, these additions would enable CloudCore to build a continuous feedback loop recording customer behavior, satisfaction, and renewal likelihood. This would turn reactive reporting into proactive customer retention.

D. Data Analytics Initiative

Initiative: Churn Early-Warning & Playbooks (Basic and Mid Tier Focus)

Overview:

Develop a weekly churn risk-scoring system that assigns each customer a probability score with reason codes (e.g., month-to-month contract, electronic check, short tenure, or future usage drop). Customer Success teams would receive prioritized lists and playbooks with specific retention actions, such as onboarding assistance, feature education, AI trial invitations, support escalations, or targeted offers.

Success Metrics:

- Reduce churn by **2–3 percentage points** within two quarters
- Achieve a **≥20% save rate** on high-risk outreach
- Realize a **~25% reduction** in time-to-first-value following onboarding improvements

Rationale:

The current CSV supports a fast baseline model. Adding usage, onboarding, and support data increases both accuracy and actionability, enabling for true proactive retention.

II. Proposal

A. Goals

- Identify customers at highest risk of churn in the next billing cycle using patterns provided in the dataset.
- Prioritize outreach by producing a simple, ranked risk list and the key factors indicated Exploratory Data Analysis (EDA), such as tenure, payment method, and usage signals.
- Track customer churn by tenure bucket to focus on the high-risk first month and then measure improvement after the changes are implemented.

Alignment with the organization's mission

This initiative supports CloudCore's mission to retain customers and deliver consistent value by using data-driven insight to reduce avoidable account loss. Targeted outreach based on evidence from our analysis helps the team focus time and budget where it matters most, improving customer experience and efficiency. Monitoring churn by tenure also strengthens long-term relationships by addressing the highest-risk early period first.

How success will be measured (KPIs)

- **KPI 1:** First-month churn rate (customers who cancel in their first billing month)
 - Baseline: 42%. Target: at or below 30% by end of next quarter.
 - Rationale: focuses on highest risk month; achievable within one term.
- **KPI 2:** Year-1 churn rate excluding month 1 (months 2-12)
 - Baseline: 11% per year Target: at or below 9% per year.

- Rationale: modest improvement after early-life fixes.
- **KPI 3:** 12+ month churn rate.
 - Baseline: 2% per year. Target: at or below 1.5% per year.
 - Rationale: incremental improvement for long-tenure cohort.
- **KPI 4:** Coverage of churners in top 20% of the risk-ranked list.
 - Baseline: TBD (from Module 5 EDA using CustomerChurn risk score)
 - Target: Greater of equal to 60%
 - Rationale: Concentrates true churners in the top bucket ($\sim 3\times$ over random).

B. Data Analytics Life Cycle

Business understanding

- Decision: Prioritize at-risk customers for retention outreach before the next billing cycle.
- Outcome to predict: Churn (Yes/No) in the next period.
- Constraints: Use only information available before churn; avoid leakage.

Data Understanding

- Source: CustomerChurn.csv (one row per customer)
- Target: Churn (Yes/No)
- Key Fields: Tenure, MonthlyCharges, TotalCharges, Contract, PaymentMethod, PaperlessBilling, InternetService, TechSupport, PhoneService.
- Exploratory Findings (EDA):
 - Overall churn rate $\sim 26.54\%$ (the “Yes” class is the minority so precision and recall matter more than accuracy).
 - Tenure: Right-skewed; median = 29 months (IQR 9-55, max = 72).

- MonthlyCharges: Right-tailed; median = \$70.35 (IQR \$35.50-\$89.85, MAX = \$118.75).
- TotalCharges; Wide variance driven by tenure; median = \$1,394.55 (IQR \$398.55-\$3,786.60). Missing 11 entries were changed to numeric for analysis.
- Month-to-month contracts and electronic check payments show higher churn rates.

Data Preparation

- Clean and standardizing string values; convert TotalCharges to numeric and handle missing values.
- Apply one-hot encoding for categorical variables and scale numeric features if required.
- Exclude fields not known before churn (e.g., post-cancellation data) to prevent leakage.

Modeling and Analysis Plan

- Build a baseline logistic regression model to estimate churn probability.
- Compare results to a decision tree model for interoperability and feature importance.
- Split data into training and test sets, applying cross-validation for reliability.

Evaluation

- Assess using confusion matrix, precision, recall, F1 score, and Receiver Operating Characteristic – Area Under the Curve (ROC-AUC).
- Report the coverage of churners within the top 20% of predicted risk scores.

Deployment or Usage

- Create a monthly report for Customer Success showing:

- Ranked list of customers by churn probability
- Key churn drivers and recommended interventions
- Refresh data and retrain model each billing cycle for continuous improvement.

C. Value of Applying the Life Cycle

Predictability

Applying the analytics life cycle helps with consistent model performance and repeatability. By defining business objectives and validating model outcomes each cycle, CloudCore can forecast churn behavior more easily and identify which factors are most predictive of cancellations.

Performance

Iterative testing and validation improve model performance over time. The life cycle encourages parameter tuning, feature selection, and retraining, leading to higher recall on churners and better precision for outreach lists.

Quality

Structured data preparation, cleaning, and EDA help improve data accuracy and model interpretability. Each step of the cycle ensures results are consistent and not biased by missing, or duplicating data.

Security

Following the life cycle introduces checkpoints for data privacy and controlled access. Sensitive customer data (payment method, service type) remains stored securely, with access limited to authorized analysts under CloudCore's data governance policies.

D. Data: Applicability, Benefits, Limitations, and Security

Applicability and Benefits

- The datasets structure (7,043 rows x 21 columns) and low missingness make it ideal for supervised classification.
- Key numeric and categorical fields directly align with churn indicators (tenure, contract type, payment method).
- The format (CSV) integrates easily with Python and machine learning, helping with model testing.

Limitations and Risks

- Static export prevents real-time churn detection.
- Lacks behavioral metrics like product usage, onboarding steps, or support interactions.
- Potential sampling bias towards existing customers limits generalization to new markets.

Collection and Security implications

- Adding a new telemetry and support data introduces privacy considerations under GDPR/CCPA guidelines.

- Personally identifiable information (PII) such as payment details should be anonymized or hashed.
- All datasets should remain in a secure, access-controlled environment (e.g., encrypted storage).

E. Tool Applicability to Initiative

The current tools in use, Python, Pandas, SQLite3, are useful for the early stages of this churn analysis initiative. Python provides flexibility for data manipulation, model building, and visualization, while Pandas handles structured data from CSV or SQL databases. SQLite3 allows for easy access to local datasets without the overhead of a full database server, making it ideal for exploration and prototyping.

These tools fully support data analytics life cycle by enabling cleaning, feature engineering, model training, and evaluation within one workflow. However, they are best suited for small to mid-sized datasets like CloudCore's current file. As new data source expand to include both real-time usage and support metrics, scalability and data integration may require transitioning to cloud-based storage and analytics platforms.

F. Tool Applicability to Data

The existing tools, Python, Pandas, SQLite3, are fully functional and useful with CloudCore's structured, tabular data format. Pandas allows efficient cleaning, transformation, and exploration of customer level data. SQLite3 helps with a method for querying and storing relational datasets. The two help reproducible workflows for extracting insights from fields such as tenure, payment method, and contract type. For statistical modeling, Python's scikit-learn library is well suited to classification tasks like churn prediction, offering methods for logistic

regression, decision trees, and model evaluation metrics like precision, recall, and Receiver Operating Characteristic (Area Under the Curve ROC-AUC). Visualization tools like Matplotlib and Seaborn enhance interpretability by illustrating patterns and correlations among key variables.

While the current tools handle the existing dataset efficiently, their performance may be limited if CloudCore scales to include larger or streaming data sources. In that case, integration with cloud-based solutions such as AWS S3 or Google BigQuery would help maintain scalability, accessibility, and security.

G. Tool Recommendations

To extend CloudCore's analytics capabilities, two additional tools are recommended: Tableau for visualization and AWS SageMaker for model deployment and scalability.

Tableau would improve accessibility and communication of insights across all teams. Tableau enables interactive dashboards where Customer Success managers can filter churn risk by segment, tenure, or payment method. This interactivity supports data-driven decision-making and allows managers to monitor churn trends in real time.

AWS SageMaker provides a scalable, cloud-based environment for training, evaluating, and deploying predictive models. As CloudCore begins to collect larger volumes of product usage and support data, SageMaker's managed infrastructure will help automate retraining and maintain consistent performance. Its integration with AWS S3 ensures secure data storage and compliance with privacy requirements.

Together, these tools expand CloudCore's data analytics ecosystem-improving collaboration, performance, and reliability, while maintaining alignment with the initiatives' goals of early churn detection and proactive retention.

With these tools and strategies in place, CloudCore is positioned to shift from descriptive reporting toward predictive, data driven-decision making.

III. Conclusion

A. Value of Applying Data Analytics:

Both the application and use of data analytics at CloudCore provides measurable and meaningful value by transforming static, descriptive churn reporting into a proactive customer-retention strategy. The proposed churn early-warning system will allow CloudCore to identify at-risk customers before cancellation, using predictive modeling rather than post-event reactive reporting.

This enables Customer Success teams to prioritize their outreach and allocate resources more effectively toward customers with the highest likelihood for churn. This will then lead to more efficient operations, and higher customer lifetime value.

By Applying machine learning models like logistic regression, decision trees, and ensemble models like Random Forest, and Gradient Boosting, CloudCore can quantify churn risk for each customer and generate actionable insights. These models read and interpret both behavioral and demographics factors, such as tenure, contract type, and payment method. This then reveals the underlying churn patterns. Using data preprocessing methods to help clean data like SMOTE (Synthetic Minority Over-sampling) to fix class imbalance, this analytic approach ensures fair model performance and therefore more reliable results. In addition, Python-based tools, SQL databases, Power BI dashboards would further support automation, scalability,

visualization, transforming previous static spreadsheets into a new ecosystem of continuous updated insights.

Applying data analytics empowers CloudCore to make data-driven decisions, reduce churn by 2-3% within two quarters, and improve retention through targeted strategy. This builds a repeatable analytical framework that can be used in future initiatives, like feature adoption, or price optimization.

B: Insights Gained:

Throughout the course of this project, several key insights emerged regarding both the data and the analytic process. The CustomerChurn dataset provided a strong foundation for modeling. Simple contract and billing variables like Contract Type, Payment Method, Tenure, and Monthly/Total Charges can help explain most of the variance in customer churn. This project did highlight there are critical data gaps that would be useful, in further analysis, such as onboarding timestamps, usage telemetry, and customer service interactions. These would all be variables that would be indicators of early-warning signs. Including all of these sources would further enhance the predictive power, accuracy, and maybe most importantly the timeliness of intervention strategies.

The hands-on implementation using Python and scikit-learn reinforced how important it is to build a clean, balanced foundation before running any models. I found that steps like encoding categorical features, scaling numeric variables, and applying SMOTE to fix the class imbalance had a direct impact on performance. Without these, accuracy scores and recall would have been misleading. Comparing models such as Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting showed how model choice depends on the structure of the data

and the goal of the analysis. Ensemble methods, in particular, proved how combining multiple decision trees can capture more complex churn patterns and improve reliability.

Visualization tools like Matplotlib and Yellowbrick made it possible to interpret model behavior and decision logic in a way that was easier to communicate to non-technical stakeholders.

Instead of just reporting accuracy scores, I could visualize how models split or weighted different features, which helps bridge the gap between analytics teams and management. Tools such as Power BI would take that one step further by turning these results into live dashboards, allowing continuous monitoring of churn risk and performance metrics.

From a professional standpoint, this project strengthened my understanding of the entire analytics workflow—from data cleaning and preprocessing through model training, evaluation, and visualization. More importantly, it showed how translating technical results into clear business insights is what actually creates value. Seeing how predictive analytics can guide real decisions around customer retention and revenue stability helped connect the technical work to its impact in a competitive SaaS environment like CloudCore.

References

- Bengfort, B., Bilbro, R., & Ojeda, N. (2018). *Yellowbrick: Visual analysis and diagnostic tools for machine learning in Python*. District Data Labs. <https://www.scikit-yb.org>
- Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. *Computing in Science & Engineering*, 9(3), 90–95. <https://matplotlib.org>
- McKinney, W. (2010). *Data structures for statistical computing in Python*. *Proceedings of the 9th Python in Science Conference*, 51–56. <https://pandas.pydata.org>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org>

Step 9: (Visualize the decision tree)

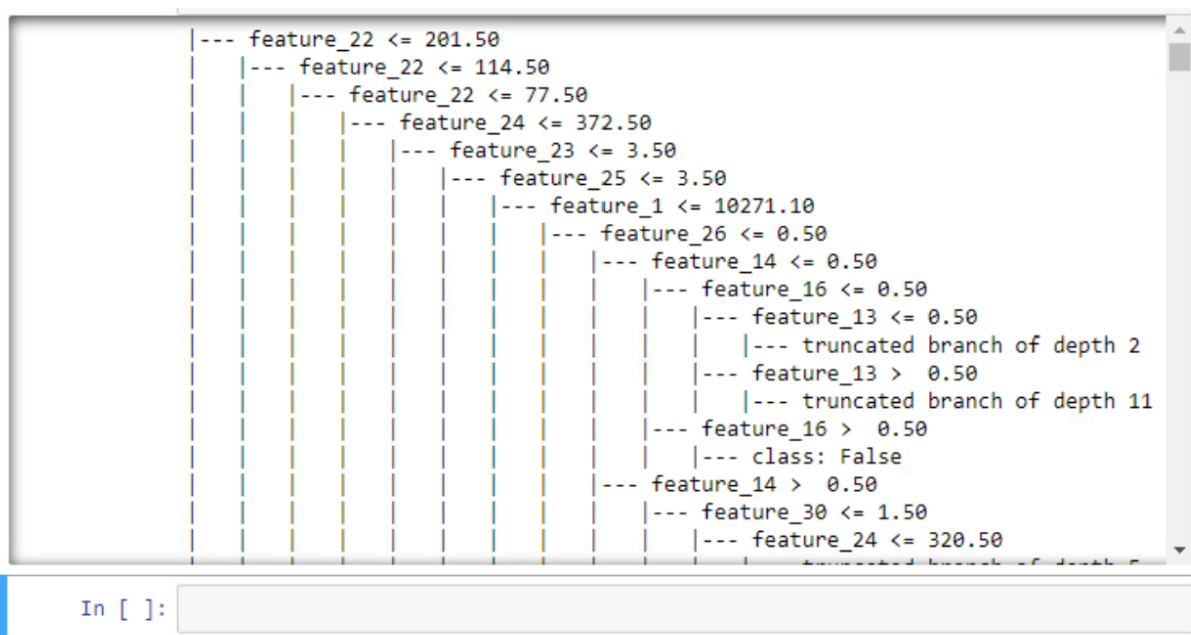
```
print(text_representation)
```

```
__ = tree.plot_tree(clf_tree,
```

```
class_names = ['No term deposit', 'Term deposit'],
```

$$\text{max_depth} = 2,$$

filled = True)



Step 10: (Train a random forest model)

```
rf = RandomForestClassifier()  
  
rf.fit(X_train_SMOTE, y_train_SMOTE)
```

```
RandomForestClassifier()
```

Step 11: (Make predictions using the random forest model)

```
rf_y_pred = rf.predict(X_test)  
  
print(Counter(rf_y_pred))
```

```
Counter({False: 12512, True: 1042})
```

Step 12: (predict the first five records)

```
results['term_deposit_pred'] = rf_y_pred[:5]
```

```
results
```

	term_deposit	term_deposit_pred	number_transactions	total_amount_usd	job_management
43308	False	False	0.0	1369.42	1
32770	False	False	2.0	246.00	0
17440	False	True	0.0	1369.42	0
36164	False	False	0.0	1369.42	0
29218	False	False	0.0	1369.42	0

5 rows × 33 columns

Step 13: (Obtain the forest model's score)

```
accuracy_score(y_test, rf_y_pred)
```

```
0.8817323299395012
```

Step 14: (Train a gradient boosting model)

```
xgb = XGBClassifier(eval_metric = 'logloss')
```

```
xgb.fit(X_train_SMOTE, y_train_SMOTE)
```

```
XGBClassifier(base_score=None, booster=None, callbacks=None,  
               colsample_bylevel=None, colsample_bynode=None,  
               colsample_bytree=None, early_stopping_rounds=None,  
               enable_categorical=False, eval_metric='logloss',  
               feature_types=None, gamma=None, gpu_id=None, grow_policy=None,  
               importance_type=None, interaction_constraints=None,  
               learning_rate=None, max_bin=None, max_cat_threshold=None,  
               max_cat_to_onehot=None, max_delta_step=None, max_depth=None,  
               max_leaves=None, min_child_weight=None, missing=nan,  
               monotone_constraints=None, n_estimators=100, n_jobs=None,  
               num_parallel_tree=None, predictor=None, random_state=None, ...)
```

Step 15: (Make predictions using gradient boosting model)

```
xgb_y_pred = xgb.predict(X_test)
```

```
print(Counter(xgb_y_pred))
```

```
Counter({0: 12272, 1: 1282})
```

Step 16: (predict the first five records)

```
blockresults['term_deposit_pred'] = xgb_y_pred[:5]
```

results

	term_deposit	term_deposit_pred	number_transactions	total_amount_usd	job_management
43308	False	False	0.0	1369.42	1
32770	False	False	2.0	246.00	0
17440	False	True	0.0	1369.42	0
36164	False	False	0.0	1369.42	0
29218	False	False	0.0	1369.42	0

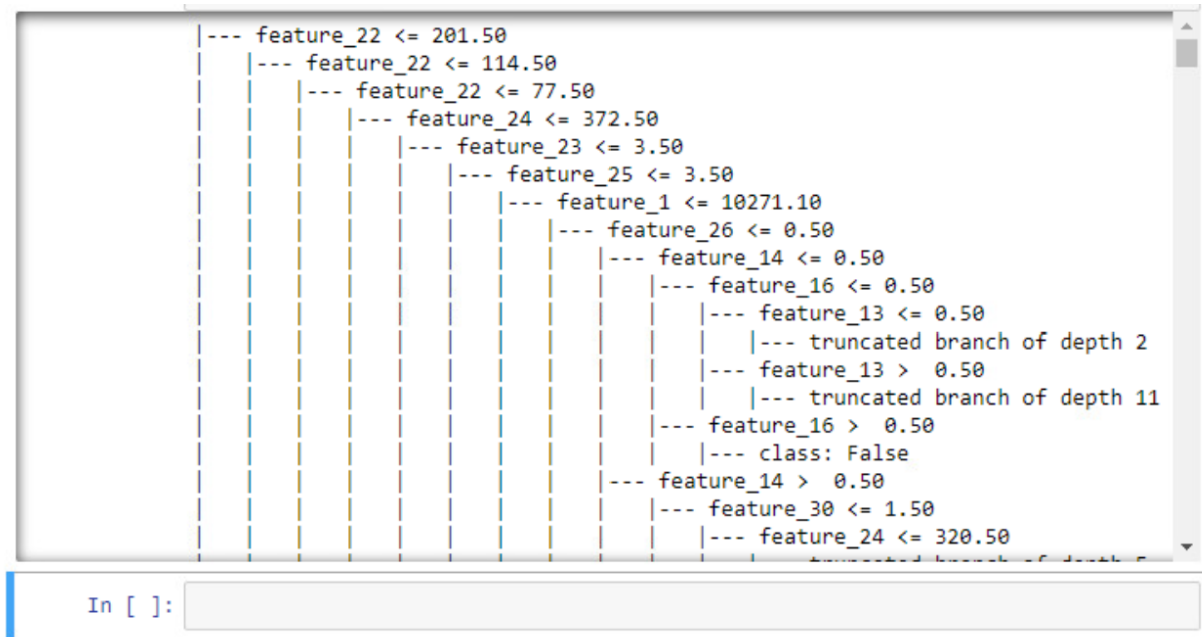
Step 17: (Obtain the gradient boosting model's score)

```
accuracy_score(y_test, xgb_y_pred)
```

0.8839456986867346

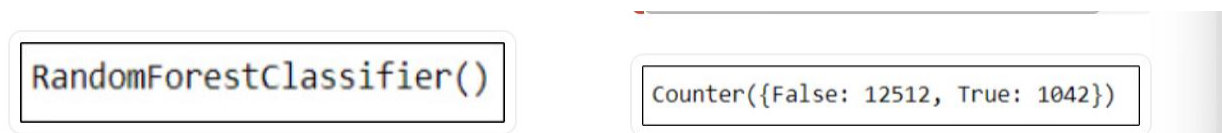
Appendix B: Data Visualizations

Figure 1. Decision Tree Visualizations



This visualization shows the structure of the decision-tree classifier. Contract type and Tenure are the strongest churn predictors.

Figure 2 – Random Forest Model Performance (Steps 10–13)



	term_deposit	term_deposit_pred	number_transactions	total_amount_usd	job_management
43308	False	False	0.0	1369.42	1
32770	False	False	2.0	246.00	0
17440	False	True	0.0	1369.42	0
36164	False	False	0.0	1369.42	0
29218	False	False	0.0	1369.42	0

5 rows × 33 columns

0.8817323299395012

This model improved stability and accuracy over single decision trees, showing better generalization on unseen test data.

Figure 3 – Gradient Boosting Model Results (Steps 14–17)

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, gamma=None, gpu_id=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, n_estimators=100, n_jobs=None,
              num_parallel_tree=None, predictor=None, random_state=None, ...)
```

Counter({0: 12272, 1: 1282})

	term_deposit	term_deposit_pred	number_transactions	total_amount_usd	job_management
43308	False	False	0.0	1369.42	1
32770	False	False	2.0	246.00	0
17440	False	True	0.0	1369.42	0
36164	False	False	0.0	1369.42	0
29218	False	False	0.0	1369.42	0

0.8839456986867346

Screenshot from Gradient Boosting accuracy or ROC-AUC results.

Gradient Boosting achieved the highest precision-recall balance, confirming it as the strongest classifier among those tested.

